

OOP Analysis of pandas

Python Library — Real World Object-Oriented Design Study

Group Members

M. Sami | M. Shzaib | M. Dawood

BS Data Science — 2nd Semester | May 2026

Library

pandas

Course

OOP — BS Data Science

Marks

100

1. Library Overview

When we started this project and looked at the library options, pandas stood out straight away. We use it in almost every data science lab but never really understood how it is built on the inside. This project gave us that chance.

pandas is a free, open-source Python library used for working with data in table form. It was created in 2008 by Wes McKinney who was working at a finance company and was frustrated with how slow data analysis was in Excel. He built pandas to solve his own problem, and today it is one of the most downloaded Python packages in the world.

The two main structures pandas gives you are the **DataFrame** (a table with rows and columns, like a spreadsheet inside Python) and the **Series** (a single column with labels). These two structures alone can handle almost every real data task you will ever need.

What pandas is used for:

- Loading CSV, Excel, and JSON files into Python in one line of code
- Cleaning messy data — removing empty rows, fixing wrong values, renaming columns
- Filtering data — for example, show only students who scored more than 70
- Grouping data — find average marks per department or city
- Merging two tables — like joining two Excel sheets by a common column
- Time-series analysis — daily stock prices, temperature readings over months
- Preparing data before feeding it into machine learning models

Installation:

Installing pandas takes just one command:

```
pip install pandas

# If you use Anaconda:
conda install pandas

# Standard import used by everyone:
import pandas as pd
import numpy as np
```

2. Class Hierarchy Diagram

We went to the actual pandas GitHub repository and read the source code files: generic.py, frame.py, series.py, and the indexes folder. What we found was a well-designed class family tree. The UML diagram image is in the /diagrams folder of our GitHub repo.

Class Relationships:

Class	Inherits From / Role
PandasObject	Top of everything. Provides <code>__repr__</code> , <code>__str__</code> , and basic utilities
NDFrame	Inherits PandasObject. Shared base for DataFrame and Series. Has <code>head()</code> , <code>tail()</code> , <code>copy()</code> , <code>describe()</code>
DataFrame	Inherits NDFrame. 2D table. Adds <code>merge()</code> , <code>pivot()</code> , <code>groupby()</code> , <code>sort_values()</code> , <code>drop()</code>
Series	Inherits NDFrame + IndexOpsMixin. 1D labeled array. Adds <code>value_counts()</code> , str accessor, cat accessor
Index	Base class for all index types. Provides label-based lookup, <code>union()</code> , <code>intersection()</code>
RangeIndex	Inherits Index. Default integer index (0,1,2,3...). Very memory efficient
DatetimeIndex	Inherits Index. For date and time based indexing. Supports <code>resample()</code> , <code>shift()</code>
MultiIndex	Inherits Index. Multi-level hierarchical labels like (City, Year)
GroupBy	Used by DataFrame and Series. Handles split-apply-combine operations

The key design idea is that NDFrame is like an abstract middle layer. Nobody creates an NDFrame object directly — you always use DataFrame or Series. NDFrame defines what all pandas objects can do, and the subclasses fill in the specific details. This is the Template Method design pattern.

3. OOP Principles in pandas

In class we studied four OOP principles. When we read the pandas source code, we found real examples of all four of them. Here we explain each one in simple words with actual working code.

3.1 Encapsulation — Hide the Messy Parts

Encapsulation means keeping complicated internal stuff private and only showing the user what they need to see. In Python, private things use an underscore prefix like `_data`.

The best example in pandas is how it stores data internally. Inside every `DataFrame` there is a private attribute called `_data` — a `BlockManager` object that manages how data sits in memory. Users are never supposed to touch it directly. Instead pandas gives clean public methods like `.values`, `.dtypes`, `.shape`, and `.to_numpy()`.

```
import pandas as pd

df = pd.DataFrame({'Name': ['Sami', 'Shzaib', 'Dawood'], 'Marks': [88, 76, 92]})

# Encapsulation – use the PUBLIC interface (the safe way)
print(df.shape) # (3, 2) – safe access to dimensions
print(df.dtypes) # shows column types safely
print(df.to_numpy()) # safe extraction of raw values

# df._data <-- PRIVATE – we should never touch this directly
# pandas changed _data completely in version 2.0
# Because it was private, nobody's code broke!
```

■ *pandas completely changed its internal `_data` storage in version 2.0. Because users always used the public API, not `_data` directly, no user code broke. That is the whole point of encapsulation.*

3.2 Inheritance — Children Get Parent's Abilities

Inheritance means a child class automatically gets all methods and attributes of its parent. This saves code because you do not write the same thing twice.

The clearest example: both `DataFrame` and `Series` inherit from `NDFrame`. Methods like `head()`, `tail()`, `describe()`, `fillna()`, `copy()` are written only ONCE inside `NDFrame`. Both `DataFrame` and `Series` get them for free, with no extra code.

```
import pandas as pd

df = pd.DataFrame({'A': [10, 20, 30]})
s = pd.Series([100, 200, 300])

# head() is NOT written separately in DataFrame or Series
# It is inherited from NDFrame – written only once!
print(df.head(2)) # works on DataFrame
print(s.head(2)) # works on Series too
```

```
# Check the inheritance chain (MRO = Method Resolution Order)
print(pd.DataFrame.__mro__)

# (DataFrame, NDFrame, PandasObject, object)

print(pd.Series.__mro__)

# (Series, IndexOpsMixin, NDFrame, PandasObject, object)

# Notice: Series uses MULTIPLE inheritance (two parents)!
```

■ *Series uses multiple inheritance — it gets NDFrame methods AND IndexOpsMixin methods (like `argmax`, `argmin`, `value_counts`). DataFrame does not need those, so they are only added to Series. Smart design!*

3.3 Polymorphism — Same Name, Different Result

Polymorphism means the same method or operator behaves differently depending on what type of object you use it with. One name, many forms.

The `.apply()` method is the best example. You can pass it any kind of callable — a numpy function, a lambda, a built-in, or a custom class — and it works correctly every single time:

```
import pandas as pd
import numpy as np

df = pd.DataFrame({'marks': [85, 92, 61, 78, 45]})

# Same .apply() – works with COMPLETELY different things:
df['sqrt'] = df['marks'].apply(np.sqrt) # numpy function
df['double'] = df['marks'].apply(lambda x: x * 2) # lambda
df['grade'] = df['marks'].apply(
    lambda x: 'A' if x>=85 else 'B' if x>=70 else 'C' if x>=55 else 'F'
)
print(df)

# Operator overloading is also polymorphism:
s1 = pd.Series([1, 2, 3], index=['a', 'b', 'c'])
s2 = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
print(s1 + s2) # aligns by index label then adds – not normal Python + !
```

3.4 Abstraction — Simple Outside, Complex Inside

Abstraction means hiding the complicated implementation behind a simple, clean interface. The user just calls a method and gets a result — they do not need to know how it works internally.

```
import pandas as pd

df = pd.DataFrame({
    'student': ['Sami', 'Shzaib', 'Dawood', 'Ali', 'Sara'],
```

```
'marks': [88, 76, 92, 65, 80],
'dept': ['CS', 'CS', 'DS', 'DS', 'CS']
})

# ONE LINE – hides massive internal complexity:
print(df.describe())

# Internally: loops every column, checks dtype, computes count/mean/
# std/min/25%/50%/75%/max, aligns all results – user sees nothing of this

# groupby also hides the entire split-apply-combine pattern:
result = df.groupby('dept')['marks'].mean()

# Internally: hash-based grouping, iteration, aggregation, combining
print(result)
```

■ *Before pandas, a groupby operation needed 20+ lines of Python loops and dictionaries. Now it is one method call. That is abstraction doing its job perfectly.*

4. Design Decision Critique — Copy-on-Write

One of the most important design decisions in recent pandas history is Copy-on-Write (CoW), which became the default in pandas 2.0. This fixed a problem that confused beginners and experts alike for many years.

The Old Problem (before pandas 2.0):

When you sliced a DataFrame, you might get either a view (a reference to the same memory) or a copy (new memory), and it was completely unpredictable which one you got. Modifying the result might or might not change the original:

```
df2 = df[['Name', 'Marks']] # is this a VIEW or a COPY?
df2['Marks'] = 0 # does this change the original df?
# Answer: NOBODY KNEW — it depended on pandas internals
# This caused the famous SettingWithCopyWarning that confused everyone
```

The New Solution — Copy-on-Write:

With CoW the rule is simple: no operation will ever silently modify another object. If you try to modify a DataFrame that came from another DataFrame, pandas automatically makes a copy first. The original is always protected.

Advantage	Disadvantage
No more silent mutations of original data	Code written for pandas 1.x may behave differently
SettingWithCopyWarning completely gone	Users need to learn the new mental model
Actually faster — fewer unnecessary copies	Small extra memory to track shared objects
Behaviour is now 100% predictable	Migration from 1.x to 2.x needs some effort

Our view: Copy-on-Write was absolutely the right decision. The old behaviour was genuinely confusing and caused real bugs in real production code. Polars (the main alternative) made all DataFrames completely immutable from day one, which avoids the problem entirely but forces a very different programming model. pandas found a good middle ground — keep the familiar API but make it predictable.

5. Comparison — pandas vs Polars

We also looked at Polars, a newer library that does the same job as pandas but is built with a completely different design philosophy. Here is what we found:

Feature	pandas	Polars
Language	Python + C (NumPy)	Rust — very fast at core level

OOP Design	Deep hierarchy: NDFrame → DataFrame/Series	Flat — no shared base class
Mutability	Mutable (CoW in v2.0)	Immutable — every operation returns new object
Inheritance	Heavy — 10+ Index subclasses	Minimal — prefers composition
Polymorphism	.apply() accepts any callable	.map() replaces apply()
Performance	Good with PyArrow backend	Faster — multi-threaded by default
Ecosystem	Huge — every DS tool supports it	Growing fast but still newer
Learning	Large community, easy to find help	Different syntax, steeper curve

The biggest OOP difference is the philosophy. pandas was built with a rich class hierarchy because that made sharing code easy between DataFrame and Series. Polars was designed from scratch in 2020 with a different goal — maximum speed with no Python overhead — so it avoids deep class hierarchies entirely. For beginners, pandas is much easier. But Polars is worth learning when performance matters.

6. Custom Extension — StudentDataFrame

For the coding part we subclassed pandas DataFrame to create a StudentDataFrame. We made it relevant to us as students — a grade management tool. It adds three new methods that a normal DataFrame does not have.

```
import pandas as pd

class StudentDataFrame(pd.DataFrame):
    """
    Custom extension of pandas DataFrame for student grade management.
    Inherits all DataFrame functionality and adds education-specific methods.
    Made by: M.Sami, M.Shzaib, M.Dawood – BS Data Science 2026
    """

    # REQUIRED for pandas subclassing:
    # Tells pandas to return StudentDataFrame (not plain DataFrame)
    # after any operation like copy(), slice, groupby etc.
    @property
    def _constructor(self):
        return StudentDataFrame

    def add_grades(self):
        """New method: adds a grade column based on marks (0-100)."""
        if 'marks' not in self.columns:
            raise ValueError("Need a 'marks' column!")

        def to_letter(m):
            if m >= 85: return 'A'
            elif m >= 70: return 'B'
            elif m >= 55: return 'C'
            elif m >= 40: return 'D'
            else: return 'F'

        result = self.copy() # inherited from NDFrame
        result['grade'] = result['marks'].apply(to_letter) # inherited apply()
        return result

    def class_stats(self):
        """New method: returns a summary of class performance."""
        if 'marks' not in self.columns:
            raise ValueError("Need a 'marks' column!")
        return {
            'Average Marks' : round(self['marks'].mean(), 1),
            'Highest Marks' : self['marks'].max(),
            'Lowest Marks' : self['marks'].min(),
```


- [3] NDFrame source — [pandas/core/generic.py](#) on GitHub
- [4] DataFrame source — [pandas/core/frame.py](#) on GitHub
- [5] Series source — [pandas/core/series.py](#) on GitHub
- [6] Polars Documentation — [docs.pola.rs](#)
- [7] Copy-on-Write explanation — [pandas.pydata.org/docs/whatsnew/v2.0.0.html](#)
- [8] Python for Data Analysis by Wes McKinney — O'Reilly Media (3rd Edition)
- [9] Python OOP Tutorial — [docs.python.org/3/tutorial/classes.html](#)
- [10] Real Python pandas Guide — [realpython.com](#)

M. Sami | M. Shzaib | M. Dawood — BS Data Science 2nd Semester — OOP Final Project — May 2026